

APPENDIX A SUPPLEMENTARY CASE STUDIES

This appendix retains five compact cases that support Section IX without interrupting the main narrative. The factorial and GCD cases show successful inductive-helper synthesis; MulLoop shows sign-sensitive invariant synthesis; Ackermann marks a recursion pattern beyond the current ACSL/SMT automation envelope; and matrix multiplication shows a verifier-timeout boundary rather than a generation failure.

a) *Factorial: Guarded Inductive Helper.*: Fig. 12 complements the phase-split loop case in Section IX: the loop may be skipped when $n < 1$, and the entered region requires the inductive relation $f == \text{product}(1, i - 1)$. Symbolic execution provides the entered/unentered guard, while the static route selects the inductive-helper template that supplies the ACSL `product` definition used by both the invariant and the postcondition.

b) *GCD: Recursive Summary with a Simple Measure.*: Fig. 13 shows the recursion case where the current workflow succeeds cleanly. When symbolic execution reaches a recursive call, SESPEC switches to the recursion template, emits `gcd_logic` branch-by-branch, and uses `decreases a + b`. This case is useful mainly as a contrast to Ackermann in Appendix A-0d: GCD needs only a single-measure descent and a single-unfold style of reasoning.

c) *MulLoop: Signed Multiplication Loop.*: Fig. 14 computes $a \cdot b$ by addition or subtraction depending on the sign of b . The negative branch requires the asymmetric invariant `res == -i * a`; reusing the positive-branch form would make the functional postcondition fail. In our run, SESPEC retains `ensures \result == a * b` because the branch condition selects the correct invariant template.

Supplementary Case: guarded factorial helper

```
int unknown();

/*@
  logic integer product(integer begin, integer end) =
    (end < begin) ? 1 : product(begin, end - 1) * end;
*/

/*@
  requires \true;
  assigns \nothing;
  ensures (n < 1) ==> \result == 1;
  ensures (n >= 1) ==> \result == product(1, n);
*/
int factorial9(int n) {
  int i = 1;
  int f = 1;

  /*@
    loop invariant (1 <= \at(n,Pre)) ==> (1 <= i <= n + 1);
    loop invariant (1 <= \at(n,Pre)) ==> (f == product(1, i - 1));
    loop invariant (! (1 <= \at(n,Pre)) ==> ((f == 1) && (i == 1) && (n == \at(n,
      Pre))));
    loop invariant n == \at(n,Pre);
    loop assigns f, i;
  */
  while (i <= n) {
    f = f * i;
    i = i + 1;
  }

  return f;
}

void goo9() {
  int t = factorial9(5);
  /*@ assert t == 120;
*/
}
```

Fig. 12. Supplementary factorial case: a guarded loop invariant with a recursive product helper.

d) *Ackermann: Recursion Beyond ACSL/SMT Automation.*: Fig. 15 marks a boundary rather than a prompt failure. The generated helper follows the C branch structure, but Ackermann needs a lexicographic termination argument and nested induction that are not handled well by the current ACSL/SMT stack. A scalar decreases $m * 1000 + n$ is not a sound substitute for the true lexicographic descent once n grows, and the functional postcondition would require proof support beyond the single-unfold reasoning that succeeds for GCD.

Supplementary Case: recursive GCD summary

```
/*@
  logic integer gcd_logic(integer a, integer b) =
    (a == 0) ? b :
    (b == 0) ? a :
    (a == b) ? a :
    (a > b) ? gcd_logic(a - b, b) :
    gcd_logic(a, b - a);
*/

/*@
  requires \true;
  decreases a + b;
  assigns \nothing;
  ensures \result == gcd_logic(a, b);
*/
int gcd(int a, int b)
{
  if (a == 0)
    return b;

  if (b == 0)
    return a;

  if (a == b)
    return a;

  if (a > b)
    return gcd(a - b, b);
  return gcd(a, b - a);
}

int main2()
{
  int a = 98, b = 56;
  int c = gcd(a, b);
  /*@ assert c == 14;
*/
  return 0;
}
```

Fig. 13. Supplementary recursive GCD case: a recursive logic summary with a single arithmetic termination measure.

Supplementary Case: signed multiplication loop

```
int foo57(int a, int b);

/*@
  requires \true;
  assigns \nothing;
  ensures \result == a * b;
*/
int foo57(int a, int b) {
  int res = 0;
  if (b >= 0) {
    /*@
      loop invariant 0 <= i <= b;
      loop invariant res == i * a;
      loop assigns i, res;
      loop variant b - i;
    */
    for (int i = 0; i < b; i++) {
      res = res + a;
    }
  } else {
    /*@
      loop invariant 0 <= i <= -b;
      loop invariant res == -i * a;
      loop assigns i, res;
      loop variant -b - i;
    */
    for (int i = 0; i < -b; i++) {
      res = res - a;
    }
  }
  return res;
}
```

Fig. 14. Supplementary MulLoop case from SpecGenBench.

e) *Matrix Multiplication: Triple-Nested Loops Beyond Per-Loop Symbolic Context.*: Fig. 16 is the multi-loop boundary that Reviewer 2’s failure-case question targets. SESPEC emits the intended quantified postcondition and the matching outer-loop invariants, while the innermost `s == mat_sum(..., k)` invariant is discharged by one-step induction. The remaining quantified obligations time out in Z3 because they combine recursive logic unfolding, havoc-updated array memory, and non-linear index arithmetic; refinement therefore strips the unproved clauses and the accepted

specification collapses to verifier-valid but vacuous ensures `\true`. This case illustrates a proof-automation limit rather than a generation failure.

APPENDIX B FULL PROMPT TEMPLATES

This appendix lists the templates used by the main LLM call sites referenced in Section VI-B, abridged to their static skeleton and runtime slots (the verbose in-prompt admonitions are elided). Each template’s static body is shown in black; runtime-injected slots appear in gray with colored tags from the shared figure palette: **LOOP/SYMBOLIC** for loop-aware symbolic context, **ROUTE/CALL** for route and call-site context, and **VERIFIER/SPECIFICATION** for verifier errors and specification edits.

A. Loop-invariant generation

The example below is a filled loop-invariant prompt for a small branch-then-loop program, with the long example and guidance blocks elided.

Supplementary Case: Ackermann recursion boundary

```
int fool(int m, int n);

/*@
  logic integer ackermann(integer m, integer n) =
    (m == 0) ? n + 1 :
    (n == 0) ? ackermann(m - 1, 1) :
    ackermann(m - 1, ackermann(m, n - 1));
*/

/*@
  requires m >= 0 && n >= 0;
  decreases m * 1000 + n;
  assigns \nothing;
  ensures \result == ackermann(m, n);
*/
int fool(int m, int n) {
  if (m == 0) {
    return n + 1;
  }
  if (n == 0) {
    return fool(m - 1, 1);
  }
  return fool(m - 1, fool(m, n - 1));
}
```

Fig. 15. Supplementary Ackermann case. The candidate specification is syntactically clean but not discharged by Frama-C/WP under the current ACSL/SMT backend.

Supplementary Case: matrix-multiplication proof boundary

```
/*@
  logic integer mat_sum(int *A, int *B, integer ca_rb, integer cb,
    integer i, integer j, integer k) =
    k <= 0 ? 0 :
    mat_sum(A, B, ca_rb, cb, i, j, k - 1) +
    A[i * ca_rb + (k - 1)] * B[(k - 1) * cb + j];
*/

/*@
  requires \valid(out + (0 .. ra*cb - 1));
  requires \valid(A + (0 .. ra*ca_rb - 1));
  requires \valid(B + (0 .. ca_rb*cb - 1));
  requires \separated(out + (0..ra*cb-1), A + (0..ra*ca_rb-1));
  requires \separated(out + (0..ra*cb-1), B + (0..ca_rb*cb-1));
  // default precondition also bounds every cell to [0,100] and each
  // dimension product to (0,100), keeping the arithmetic overflow-free
  assigns out[0 .. ra*cb - 1];
  ensures \forall integer i, j; 0 <= i < ra && 0 <= j < cb ==>
    out[i*cb + j] == mat_sum(A, B, ca_rb, cb, i, j, ca_rb); // Z3 TIMEOUT (10 s)
*/
void mat_mul(int *out, int *A, int *B, int ra, int ca_rb, int cb) {
  /*@
    loop invariant 0 <= i <= ra;
    loop invariant \forall integer ii, jj; 0 <= ii < i && 0 <= jj < cb ==>
      out[ii*cb + jj] == mat_sum(A, B, ca_rb, cb, ii, jj, ca_rb); // Z3
      TIMEOUT (10s)
    loop invariant \forall integer p; i*cb <= p < ra*cb ==> out[p] == \at(out[p], Pre); // Z3 TIMEOUT
    loop assigns i, out[0 .. ra*cb - 1];
  */
  for (int i = 0; i < ra; i++) {
    /*@
      loop invariant 0 <= j <= cb;
      loop invariant \forall integer jj; 0 <= jj < j ==>
        out[i*cb + jj] == mat_sum(A, B, ca_rb, cb, i, jj, ca_rb); // Z3
        TIMEOUT (10s)
      loop assigns j, out[i*cb .. i*cb + cb - 1];
    */
    for (int j = 0; j < cb; j++) {
      int s = 0;
      /*@
        loop invariant 0 <= k <= ca_rb;
        loop invariant s == mat_sum(A, B, ca_rb, cb, i, j, k); // DISCHARGED (
        one-step induction)
        loop assigns k, s;
      */
      for (int k = 0; k < ca_rb; k++)
        s = s + A[i*ca_rb + k] * B[k*cb + j];
      out[i*cb + j] = s;
    }
  }
}
```

Fig. 16. Supplementary matrix-multiplication case ($ra \times ca_rb$ by $ca_rb \times cb$ product). SESPEC generates the correct quantified specification, but Frama-C/WP→Z3 times out on the outer invariants and postcondition, so the accepted specification degrades to ensures `\true`.

Loop-invariant generation prompt

```
### Role / Task ###
Given a C program with a loop, generate ACSL loop invariants
that let Frama-C verify it.

ROUTE examples + ACSL helper policy for the matched route, omitted
GUIDANCE strength / supplementary / assertion guides, omitted

### Rules ###
- Only fill the loop-invariant PLACE_HOLDERS (or add a helper block
  before the function); keep existing annotations unchanged.
- Use the pre-condition, not \at(var, LoopEntry).
- Emit one self-contained ``c`` block.

SYMBOLIC STATE supplied by the symbolic executor
Pre-condition:
  (\at(a,Pre)==0 && y==0 && x==1 && a==\at(a,Pre))
  || (\at(a,Pre)!=0 && y==0 && x==0 && a==\at(a,Pre))

Loop + skeleton to complete:
/*@
  loop invariant (\at(a,Pre)!=0) ==> ((y<100000) ==>
  ↪ PLACE HOLDER x);
  loop invariant (\at(a,Pre)!=0) ==>
    (! (y<100000) ==> (y==0 && x==0 && a==\at(a,Pre)));
  loop invariant (\at(a,Pre)!=0) ==> (a==\at(a,Pre));
  ... (symmetric a==0 branch omitted) ...
*/
while (y < 100000) { x = x + y; y = y + 1; }
```

B. Precondition fallback

Precondition fallback prompt

```

### Role: ###
Generate ONLY function-level preconditions plus supporting
predicate/logic declarations needed by them.

### Goal: ###
Rule out Frama-C runtime errors:
- pointer dereferences must be covered by \valid or
  \valid_read;
- array ranges must cover all indexed accesses;
- inductive data structures use structural predicates from
  examples.

ROUTE EXAMPLES
{examples}

### Output format: ###
Output exactly one fenced ``acsl`` block:
/*@ predicate <name>(...) = ...; */
/*@ logic <type> <name>(...) = ...; */
/*@
  requires <clause>;
*/

### Hard rules: ###
- Output only ACSL annotations.
- Produce requires only; no ensures, assigns, or loop invariant.
- Each predicate or logic declaration lives in its own /*@ ... */
  ↳ block.

Function under specification:
{code}

```

C. Postcondition and assignment generation

Postcondition / assignment generation prompt

```

### Task: ###
Transform the following C code into an ACSL-annotated version.

### Rules: ###
- Only fill ensures PLACEHOLDER and assigns PLACEHOLDER.
- The input is a complete C file; callee specifications are
  read-only context.
- Output only the modified target function block. The
  harness splices it back into the original file.
- Helper predicate/logic definitions are allowed and required
  if the target specification references a name not already defined.
- Replace the ensures PLACEHOLDER with as many strong,
  ↳ non-redundant
  clauses as the behavior admits (typically 3-8): the closed-form
  result, per-branch facts, and frame consequences.
- Inline relations directly in ensures; add a predicate/logic helper
  only when the relation is recursive or higher-order.

LOOP-AWARE DERIVATION
{loop_guide}

### ACSL pitfalls ###
UNIVERSAL + SYMBOLIC-ROUTE HINTS
{category_hints}

RESOLVED ARRAY SIZES
{array_length_facts}

### C Program Code: ###
{code}

```

D. Syntax repair

A deterministic ACSL fixer (Section V) runs before this prompt and clears the common mechanical faults; the template below is the LLM fallback used only when the fixer cannot resolve the parser error.

Syntax-repair prompt

```

### Role: ###
You are an expert in C and Frama-C.

### Task: ###
Correct syntactically incorrect ACSL annotations using
Frama-C errors.

Syntax error:
VERIFIER ERROR
{error_str}

C code with incorrect ACSL annotations:
{c_code}

### ACSL pitfalls ###
UNIVERSAL + SYMBOLIC-ROUTE HINTS
{category_hints}

### Rules: ###
- Do not modify function bodies or signatures.
- Only change ACSL annotations.
- New top-level predicate/logic blocks may be added before
  the function.
- If an unbound logic/predicate name is reported, define it
  or remove every clause that references it.

```

E. Pairwise specification strength judges

Fig. 17 gives the three independent judge prompts used for Table VII. Each invocation receives only the target function's annotation blocks relevant to that dimension; the complete prompt text and structured responses are included in the artifact.

F. Verifier-guided semantic refinement

Fig. 18 gives the refinement prompt used by the verifier-guided refinement loop of Section V.

Verifier-guided refinement prompt

```

### Task: ###
Frama-C reports verification failures grouped by category. Each
↳ category
has a different fix strategy; only relevant strategies are shown.

### Categorized errors ###
ACTIVE ERROR BUCKETS
{error_str}

### Per-category fix strategy ###
ACTIVE STRATEGIES ONLY
{strategy_blocks}

### Current C code with annotations ###
{c_code}

LOOP-AWARE DERIVATION
{loop_guide}

### ACSL pitfalls ###
UNIVERSAL + SYMBOLIC-ROUTE HINTS
{category_hints}

### Output rules ###
- The target function's requires may be strengthened when necessary.
EDIT MODE
{output_format_block}
- Do not modify user assert statements.
- Do not modify correct loop invariants or correct postconditions.
- Wrap the corrected target block, or the complete file in
  ↳ full-file mode,
  in one ``c`` fenced block.

```

Fig. 18. Verifier-guided semantic refinement prompt.

APPENDIX C

JUDGE CONFIGURATION AND POSITION-BIAS AUDIT

We used GPT-5.4 with a 2,048-token output limit, no input truncation, and the provider-default temperature, top-*p*, and reasoning effort. Each precondition, postcondition, and loop-invariant comparison used a separate API call and the corresponding prompt in Appendix B-E. The expert review was conducted by two master's-level researchers with formal-verification backgrounds.

Precondition judge	Postcondition judge	Loop-invariant judge
<pre> [System] Compare only accepted input ↪ domains. A missing precondition means true. Report ↪ which predicate is WEAKER, i.e., which one accepts ↪ more inputs. Do not compare postconditions or ↪ invariants. [User] Evaluate independently: B_to_A: P_B implies P_A A_to_B: P_A implies P_B B_to_A only => A_accepts_more A_to_B only => B_accepts_more both => equivalent neither/unknown => incomparable Return both directions, the ↪ relation, and a concise implication-based reason as strict ↪ JSON. </pre>	<pre> [System] Compare only normal-return ↪ guarantees on the common accepted input domain. Missing ↪ preconditions or postconditions mean true. Ignore ↪ invariant content. [User] Let D = P_A AND P_B. Evaluate ↪ independently: A_to_B: D AND Q_A implies Q_B B_to_A: D AND Q_B implies Q_A A_to_B only => A_guarantees_more B_to_A only => B_guarantees_more both => equivalent neither/unknown => incomparable Do not reward a wider input domain ↪ or infer an unstated guarantee from the body. ↪ Return both implication directions, relation, ↪ and reason as JSON. </pre>	<pre> [System] Compare only explicit loop ↪ invariants at corresponding program points. Ignore function ↪ contracts. [User] At each point, conjoin that side's ↪ invariants. Evaluate independently at every ↪ point: A_to_B: I_A implies I_B B_to_A: I_B implies I_A A_to_B only => ↪ A_invariants_stronger B_to_A only => ↪ B_invariants_stronger both everywhere => equivalent neither/unknown => incomparable Do not merge different loop points ↪ or infer reachable states from bodies. Return both ↪ implication directions, relation, and a concise decisive ↪ reason as JSON. </pre>

Fig. 17. Independent strength-judge prompts used before expert review. A and B denote two anonymized specifications; tool identities are not disclosed to the judge.

The primary run placed the baseline in A and SESPEC in B. To test order sensitivity, we repeated every judge decision with SESPEC in A and the baseline in B, then mapped the output back to the original tool orientation. Table XII compares the raw judge results before expert correction. Pairs with no substantive predicate on either side are deterministically equal and are included in the reported agreement. The principal AutoSpec trends are stable. The SpecGen postcondition and invariant directions also persist, but its precondition direction reverses. Because Java/JML and C/ACSL have different expressive and semantic capabilities, a complete cross-language precondition comparison is not possible; this heterogeneity also amplifies positional bias in the judge.

TABLE XII
A/B-ORDER AUDIT OF THE LLM JUDGE. OUTCOME TUPLES ARE
SESPEC/BASELINE/EQUAL/INCOMPARABLE.

Baseline	Dimension	Pairs	Agreement	Primary order	Swapped order
AUTOSPEC (C/ACSL)	Preconditions	698	695 (99.6%)	96/290/311/1	99/288/310/1
	Postconditions	698	672 (96.3%)	342/18/320/18	349/11/325/13
	Loop invariants	540	435 (80.6%)	120/50/5/365	160/49/5/326
SPECGEN (Java/JML)	Preconditions	673	546 (81.1%)	161/178/221/113	200/86/232/155
	Postconditions	673	518 (77.0%)	102/246/218/107	141/203/217/112
	Loop invariants	473	382 (80.8%)	183/9/2/279	198/3/1/271

APPENDIX D TRANSLATION DETAIL

We used GPT-5.4 to translate the 250 FormalBench and 59 SpecGenBench Java programs into C; the other 91 SV-COMP cases are native C. For each Java case, the model was asked to preserve the target method's branches, loops, assignments, and returns while adapting its types and memory operations to C. We removed JML annotations and goal assertions and renamed the target function to `foo` followed by its case ID. Programs that could not be translated into the supported C fragment and compiled were excluded; for example, SpecGenBench's `ConvertToTitle` uses `Java StringBuffer.append` and `reverse` and was not included. All 400 selected C programs pass GCC C11 compilation: 309 translated programs and 91 native-C programs.

In the resulting programs, Java `int`, `long`, and `boolean` are represented by C `int`, `long`, and `0/1-valued int`. An array parameter `int[] a` becomes `int *a`, `int a_len`; `a.length` becomes `a_len`, and array lengths are passed through calls. A local `new int[n]` becomes a C allocation with a stored length. Constants such as `Integer.MIN_VALUE` and the few string operations are adapted to their C counterparts.

Compilation checks syntax, not full semantic equivalence. Cross-language comparisons are restricted to normal executions with valid memory accesses, successful allocation, and no signed overflow.

We also checked translation behavior by differential execution on all 309 Java/C pairs. The check covered 4,325 comparable executions using deterministic edge-case and seeded pseudorandom inputs, and compared the return value and any modified integer-array arguments. The Java programs were compiled for Java 8, and the C programs were compiled with GCC C11 and undefined-behavior checks. Of these executions, 4,323 produced matching observations; the other two triggered C undefined behavior in the same program. Thus, 308 of 309 pairs showed no discrepancy on their tested inputs, while one pair exposed a cross-language semantic difference.

In that pair, case 112, input `'0'` makes the expression `1 << (c - 'a')` shift by a negative count in C, whereas Java defines the shift and returns 1. This discrepancy is outside the common defined-behavior domain, but it shows why compilation and finite-input agreement cannot establish unrestricted Java/C equivalence. The differential result is empirical evidence over the tested inputs, not a proof of semantic equivalence.